


# Procesadores del Lenguaje

## Universidad Carlos III de Madrid

## Práctica 1

Curso 2025/2026

**Grupo:** 81

**Autores:**

Fran Jurado Basté (100522213)
Sergio González Jiménez (100522264)

# Índice

**1. Introducción** **2**

**2. Desarrollo de las Especificaciones** **2**

2.1. 1. Variables Globales . . . . . 2
2.2. 2. Función MAIN . . . . . 2
2.3. 3. Impresión de Cadenas (puts) . . . . . 3
2.4. 4. Impresión de Expresiones (printf) . . . . . 3
2.5. 5. Operadores, Precedencia y Asociatividad . . . . . 4
2.6. 6. Estructura de control WHILE . . . . . 4

**3. Ampliación de Especificaciones (Parte 2)** **4**

3.1. 7. Estructura de Control IF y ELSE . . . . . 4
3.2. Gestión de Variables Locales . . . . . 4
3.3. 9. Estructura de Control FOR . . . . . 5
3.4. 10. Estructura Switch/Case . . . . . 5
3.5. 11. Funciones de Usuario, Parámetros y Retorno . . . . . 6
3.6. 12. Implementación de Vectores (Arrays) . . . . . 7

# 1. Introducción

El objetivo de esta práctica ha sido ampliar un traductor básico desarrollado con herramientas de análisis léxico y sintáctico (Bison/Yacc y código C) para traducir un subconjunto del lenguaje C a notación prefija evaluable por un intérprete de Lisp.

# 2. Desarrollo de las Especificaciones

## 2.1. 1. Variables Globales

Para el desarrollo de esta especificación tuvimos que ampliar la **Sección 3** de la gramática para ello implementamos 3 reglas recursivas que concatenan múltiples asignaciones separadas por comas:

```
declaracion:  INTEGER lista_vars          { $$ = $2 ; } ;

lista_vars:   var_init                    { $$ = $1 ; }
            | lista_vars ',' var_init     { sprintf(temp, "%s %s", $1.code, $3.
              code) ;
                                            $$.code = gen_code(temp) ; } ;

var_init:     IDENTIF                     { sprintf(temp, "(setq %s 0)", $1.code)
    ;
                                            $$.code = gen_code(temp) ; }
            | IDENTIF '=' NUMBER          { sprintf(temp, "(setq %s %d)", $1.code
              , $3.value) ;
                                            $$.code = gen_code(temp) ; } ;
```

La regla **declaracion** es la que espera la palabara reservada **int** el token **INTEGER** seguida de una lista de variables.

La regla recursiva **lista_vars** es la que nos permite las declaraciones múltiples en una sola línea.

La regla **var_init** maneja los casos base.

## 2.2. 2. Función MAIN

Para esta especificación lo primero que hicimos fue añadir el token **MAIN** al array de keywords para que el analizador reconociera la palabra reservada **main**.

```
%token MAIN;
```

Para poder meter varias líneas de código dentro de la función, tuvimos que crear la regla recursiva **lista_sentencias**. Esta regla iba uniendo instrucciones separadas por saltos de línea .

```
lista_sentencias: instruccion                       { $$ = $1 ; }
                | lista_sentencias instruccion      {
                    sprintf (temp, "%s\n%s", $1.code, $2.code) ;
                    $$.code = gen_code (temp) ;
                  }
                ;
```

Creamos la regla funcion_main que esperaba exactamente la estructura MAIN () lista_sentencias. Al detectarla, cogía todo el bloque de código traducido ($5.code) y lo metía en el "molde"de Lisp usando un sprintf

```
funcion_main: MAIN '(' ')' '{' lista_sentencias '}' {
                  sprintf(temp, "(defun main ()\n%s\n)", $5.code) ;
                  $$.code = gen_code(temp) ;
              } ;
```

Por último le añadimos al axioma que un programa válido podía estar formado por sentencias globales seguidas de la nueva **funcion_main**.

```
axioma:       sentencia ';'              { printf ("%s\n", $1.code) ; }
                r_axioma                 { ; }
            | funcion_main               { printf ("%s\n", $1.code) ; }
                r_axioma                 { ; }
            ;
```

## 2.3. 3. Impresión de Cadenas (puts)

Nuevamente añadimos el token **PUTS** en el diccionario de palabras reservadas y a la tabla de **keywords**. Dado que el analizador léxico (`yylex`) ya extrae automáticamente el contenido entre comillas dobles como un token `STRING`, la traducción se redujo a embeber dicho texto en la función `print` de Lisp.

```
| PUTS '(' STRING ')'        { sprintf(temp, "(print %s)", $3.code) ;
                               $$.code = gen_code(temp) ; }
```

## 2.4. 4. Impresión de Expresiones (printf)

Para esta sección añadimos el token **PRINTF** y la palabra "printf.ª la lista de keywords en la Sección 4. Añadimos a sentencia la forma del printf e ignoramos $3.

```
| PRINTF '(' STRING ',' lista_elems ')' { $$ = $5 ; }
```

Usamos recursividad por la izquierda para poder concatenar infinitos elementos separados por comas:

```
lista_elems:  elem                  { $$ = $1 ; }
            | lista_elems ',' elem {
                  sprintf (temp, "%s %s", $1.code, $3.code) ;
                  $$.code = gen_code (temp) ;
              }
            ;
```

Implementamos la regla *elem* para aceptar indistintamente literales (`STRING`) o variables numéricas (*expresion*).

```
elem:         expresion  {
                  sprintf (temp, "(princ %s)", $1.code) ;
                  $$.code = gen_code (temp) ;
              }
            | STRING     {
                  sprintf (temp, "(princ %s)", $1.code) ;
                  $$.code = gen_code (temp) ;
              }
            ;
```

### 2.5. 5. Operadores, Precedencia y Asociatividad

Para esta sección añadmimos múltiples **tokens** nuevos(`EQ`, `NEQ`, `LEQ`, `GEQ`) y lógicos (`AND`, `OR`).

Tuvimos que definir la precedencia matemática usano las directivas %left y %right.

```
%left '<' '>' LEQ GEQ
%left '+' '-'
%left '*' '/' '%'
%left UNARY_SIGN '!'
```

Tuvimos que ampliar la regla **expresion** vara que soportara los nuevos operadores.

### 2.6. 6. Estructura de control WHILE

Para esta sección volvimos a introducir un nuevo token **WHILE** y a registrar la palabra while en la tabla de keywords del analizador léxico.

El reto de integrar el bucle `while` radicaba en que, en C, un bucle termina con una llave de cierre (}), mientras que nuestro bloque de *lista_sentencias* obligaba al uso de un delimitador `;`.

Para solucionarlo sin romper la gramática existente, se creó una regla puente *instruccion*:

```
instruccion:  sentencia ';'          { $$ = $1 ; }
            | bucle_while            { $$ = $1 ; } ;
```

La traducción al formato de Lisp se realizó de la siguiente manera:

```
bucle_while:  WHILE '(' expresion ')' '{' lista_sentencias '}' {
                  sprintf(temp, "(loop while %s do\n%s\n)", $3.code, $6.code) ;
                  $$.code = gen_code(temp) ;
              } ;
```

## 3. Ampliación de Especificaciones (Parte 2)

### 3.1. 7. Estructura de Control IF y ELSE

Para esta seccion añadimos los tokens **IF** Y **ELSE** y los registramos en keywords.

Añadimos sentencia_if a nuestra lista de instruccion (aprovechando que, igual que el while, el if termina con llaves y no con punto y coma).

```
sentencia_if: IF '(' expresion ')' '{' lista_sentencias '}' ELSE '{'
    lista_sentencias '}' {
                  sprintf(temp, "(if %s\n(progn\n%s\n)\n(progn\n%s\n))", $3.code
                      , $6.code, $10.code);
                  $$.code = gen_code(temp);
              } ;
```

### 3.2. Gestión de Variables Locales

Un problema relevante que encontramos fue el tratamiento de variables locales con el mismo nombre que variables globales. En C, es perfectamente válido declarar una variable global `x` y, dentro de `main`, declarar otra variable local `x` completamente independiente: son dos posiciones de memoria distintas.

El conflicto surge porque en Lisp, `setq` opera sobre variables globales por defecto. Traducir ambas declaraciones literalmente como `(setq x ...)` haría que la variable local machacara el valor de la global, corrompiendo el comportamiento del programa.

**Solución implementada**

Para resolver este conflicto introdujimos los siguientes elementos en el código C del traductor:

- Un array `local_vars[100][256]` que actúa como registro de las variables declaradas dentro de una función.
- Una bandera entera `in_function`, que vale `0` cuando el traductor está procesando el ámbito global, y `1` cuando está dentro del cuerpo de una función.
- Una cadena `current_func` que almacena el nombre de la función que se está traduciendo en cada momento.

**La función `get_var_name`**

El mecanismo central de esta solución es la función `get_var_name`, cuyo comportamiento es el siguiente:

1. Recibe el nombre original de la variable en C (por ejemplo, `x`).
2. Consulta `local_vars` para determinar si dicha variable ha sido declarada como local en la función actual.
3. Si la variable es **global**: devuelve el nombre original sin modificar. Lisp accederá directamente a la variable global correspondiente.
4. Si la variable es **local**: construye un nombre compuesto concatenando el nombre de la función y el de la variable, separados por un guión bajo. Por ejemplo, una variable local `x` dentro de `main` se traduce como `main_x`.

De este modo, las variables locales quedan aisladas en su propio espacio de nombres dentro de Lisp, evitando colisiones con las variables globales.

## 3.3. 9. Estructura de Control FOR

Para esta seccion añadimos los tokens **FOR INC DEC** y los registramos en keywords.

Luego creamos una regla específica para procesar los incrementos y traducirlos a matemáticas de Lisp usando setf. Usamos get_var_name() para mantener a salvo las variables locales:

```
incremento: INC '(' IDENTIF ')' {
                sprintf(temp, "(setf %s (+ %s 1))", get_var_name($3.code),
                    get_var_name($3.code)); ...
            }
```

Añadimos bucle_for a nuestra lista de instruccion

```
bucle_for: FOR '(' sentencia ';' expresion ';' incremento ')' '{'
    lista_sentencias '}' {
                sprintf(temp, "%s\n(loop while %s do\n%s\n%s\n)", $3.code, $5.
                    code, $10.code, $7.code);
                $$.code = gen_code(temp);
            } ;
```

## 3.4. 10. Estructura Switch/Case

Para esta sección añadimos los tokens `SWITCH`, `CASE`, `DEFAULT` y `BREAK`, registrándolos en la tabla de keywords del analizador léxico.

Al igual que el `if` y el `while`, el `switch` termina con llaves, por lo que se añadió *sentencia_switch* a la regla *instruccion*:

```
instruccion:  sentencia ';'       { $$ = $1 ; }
            | bucle_while         { $$ = $1 ; }
            | sentencia_if        { $$ = $1 ; }
            | sentencia_switch    { $$ = $1 ; } ;
```

La traducción del bloque `switch` aprovecha la construcción `case` nativa de Lisp, que evalúa una expresión y la compara con los valores de cada rama:

```
sentencia_switch: SWITCH '(' expresion ')' '{' lista_casos '}' {
                      sprintf(temp, "(case %s\n%s\n)", $3.code, $6.code) ;
                      $$.code = gen_code(temp) ;
                  } ;
```

Los casos individuales se acumulan mediante la regla recursiva *lista_casos*, que concatena cada bloque traducido separado por saltos de línea:

```
lista_casos:  caso_switch                { $$ = $1 ; }
            | lista_casos caso_switch    {
                  sprintf(temp, "%s\n%s", $1.code, $2.code) ;
                  $$.code = gen_code(temp) ;
              } ;
```

Cada caso traduce su número y su bloque de sentencias. El token `BREAK` es reconocido por la gramática pero no genera código Lisp, ya que la semántica de `case` en Lisp no lo requiere:

```
caso_switch: CASE NUMBER ':' lista_sentencias BREAK ';' {
                 sprintf(temp, "(%d\n%s)", $2.value, $4.code) ;
                 $$.code = gen_code(temp) ;
             }
```

## 3.5. 11. Funciones de Usuario, Parámetros y Retorno

Para esta sección añadimos los tokens `RETURN` y los registramos en la tabla de keywords. Las funciones de usuario siguen la misma filosofía que *funcion_main*, pero generalizadas para aceptar cualquier nombre y lista de parámetros.

### Definición de funciones

Se añadió al *axioma* la alternativa *funcion_user*, de forma que un programa válido puede contener cualquier número de funciones de usuario antes del `main`:

```
axioma: sentencia ';'       { printf("%s\n", $1.code) ; } r_axioma { ; }
      | funcion_main        { printf("%s\n", $1.code) ; } r_axioma { ; }
      | funcion_user        { printf("%s\n", $1.code) ; } r_axioma { ; } ;
```

La regla *funcion_user* reconoce la cabecera completa de la función, activa la bandera `in_function` y registra el nombre en `current_func` antes de procesar el cuerpo. Al cerrar la llave, genera el bloque `defun` de Lisp y limpia la tabla de variables locales:

```
funcion_user: IDENTIF '(' lista_params_def ')' '{'
                  { in_function = 1; strcpy(current_func, $1.code); }
                  lista_sentencias '}'
                  {
                      sprintf(temp, "(defun %s (%s)\n%s\n)",
                              $1.code, $3.code, $7.code) ;
                      $$.code = gen_code(temp) ;
```

```
                    in_function = 0 ; clear_locals() ;
                } ;
```

#### Parámetros formales

Los parámetros se recogen mediante la regla recursiva *lista_params_def*, que acepta cero o más parámetros de tipo `int` separados por comas. El caso vacío genera una cadena vacía para que `defun` no incluya argumentos:

```
lista_params_def: /* vacio */                    { $$.code = gen_code("") ; }
                | param_def                      { $$ = $1 ; }
                | lista_params_def ',' param_def {
                    sprintf(temp, "%s %s", $1.code, $3.code) ;
                    $$.code = gen_code(temp) ;
                } ;

param_def: INTEGER IDENTIF { $$ = $2 ; } ;
```

#### Llamadas a funciones

Para poder llamar a funciones tanto como sentencia independiente como dentro de una expresión, se añadieron las dos formas (con y sin argumentos) tanto en *sentencia* como en *operando*:

```
| IDENTIF '(' ')'            { sprintf(temp, "(%s)", $1.code) ;
                               $$.code = gen_code(temp) ; }
| IDENTIF '(' lista_args ')' { sprintf(temp, "(%s %s)", $1.code, $3.code) ;
                               $$.code = gen_code(temp) ; }
```

Los argumentos se acumulan con la regla *lista_args*, simétrica a *lista_params_def* pero aceptando expresiones en lugar de declaraciones:

```
lista_args: expresion                    { $$ = $1 ; }
          | lista_args ',' expresion     {
              sprintf(temp, "%s %s", $1.code, $3.code) ;
              $$.code = gen_code(temp) ;
            } ;
```

#### Retorno de valores

La sentencia `return` se traduce con `return-from` de Lisp, que requiere el nombre explícito de la función de la que se sale. Para ello se consulta la variable global `current_func`:

```
| RETURN expresion {
      sprintf(temp, "(return-from %s %s)", current_func, $2.code) ;
      $$.code = gen_code(temp) ;
  }
```

### 3.6. 12. Implementación de Vectores (Arrays)

Para esta sección no fue necesario añadir nuevos tokens ya que el analizador léxico (`yylex`) ya reconoce los corchetes `[` y `]` como literales individuales. Esto nos permitió extender la gramática de forma limpia sin tocar el léxico.

Se establecieron tres operaciones fundamentales sobre vectores:

### Declaración

Se amplió la regla *var_init* para reconocer la sintaxis `int id[N];` y traducirla a la función nativa de Lisp `make-array`. Al igual que con las variables escalares, se respeta el ámbito mediante `get_var_name`:

```
var_init: IDENTIF '[' NUMBER ']' {
              char *vname = $1.code ;
              if (in_function) { add_local(vname) ; vname = get_var_name(vname)
                  ; }
              sprintf(temp, "(setq %s (make-array %d))", vname, $3.value) ;
              $$.code = gen_code(temp) ;
          } ;
```

### Lectura (acceso)

El acceso a un elemento del vector se integró directamente en la regla *operando*, de forma que puede usarse en cualquier expresión. Se traduce con la función `aref` de Lisp, que acepta el array y el índice:

```
operando: IDENTIF '[' expresion ']' {
              sprintf(temp, "(aref %s %s)", get_var_name($1.code), $3.code) ;
              $$.code = gen_code(temp) ;
          } ;
```

### Escritura (asignación)

La escritura en una posición del vector se integró en la regla *sentencia*. Se combina `setf` con `aref` para modificar el elemento concreto del array en la posición indicada:

```
sentencia: IDENTIF '[' expresion ']' '=' expresion {
               sprintf(temp, "(setf (aref %s %s) %s)",
                       get_var_name($1.code), $3.code, $6.code) ;
               $$.code = gen_code(temp) ;
           } ;
```